Source-based research review

Six Further Ways to Simplify the DiophantineConsistency Lean Proofs

A scoped nonduplication audit, concrete proof interfaces, and checked mechanism prototypes

Prepared for Vladimir Reshetnikov

26 October 2026

Repository	https://github.com/VladimirReshetnikov/ProveIt
Source snapshot	`8e8b5dcb954969621c484588f0b902be1cfea7b3`
Toolchain	Lean 4.32.0; Mathlib pin `81a5d257c8e410db227a6665ed08f64fea08e997`
Comparison scope	Both current incoming ZIPs, their supporting material, and the directly linked retired DiophantineConsistency proof-simplification packet
Deliverables	This article; Lean prototypes and logs; a proposed production patch; reproducible finite carry experiments; source and prior-proposal inventories

Principal recommendation. Start with fused decoder correctness and signed congruence transport. For a broader reduction in proof maintenance, introduce constructor views with one unique-existence induction, then a definability-aware theorem for preservation by internal recursion. Treat depth-bound automation and explicit cyclic carries as separately scoped investments.

Abstract

This report proposes six directions that were not found in the audited proof-simplification packets. It deliberately excludes their existing recommendations about list lookup, assignment naturality, shared sum-of-squares proofs, generated power certificates, scanner abstractions, polynomial degree, coefficient bounds, and general proof engineering. The new directions are: a single exact graph theorem for the syntax decoder; congruence modulo a sign for Pell proofs; constructor views combined with one internal unique-existence induction; invariant preservation and fusion for definable internal recursion; compositional certificates for representation depth; and an explicit classification of cyclic carries. The first is accompanied by an applicable production patch with an 18-line local reduction. Standalone Lean prototypes check the main abstract mechanisms. The cyclic-carry direction includes a general ordinary mathematical proof and 57,540 finite raw-array/budget checks. These are proposals for integration, not a claim that the full project has been rebuilt or that all suggested replacements already compile in their production modules.

Contents

1 What was reviewed and what is actually new **3**

1.1 The source baseline . . . 3

1.2 Nonduplication audit . . . 3

1.3 Evidence levels . . . 4

2 Fuse syntax-decoder correctness into one graph theorem **4**

2.1 Concrete duplication . . . 4

2.2 The stronger public specification . . . 5

2.3 Why the proof is smaller . . . 5

2.4 Checked artifact and integration step . . . 5

3 Treat Pell sign ambiguity as a compositional relation **6**

3.1 The source target . . . 6

3.2 The proof reduction . . . 6

3.3 A precise limitation . . . 6

3.4 Checked artifact and expected payoff . . . 7

4 Constructor views and one internal unique-existence induction **7**

4.1 Expose the meaning of a known constructor once . . . 7

4.2 Strengthen the induction from totality to unique existence . . . 7

4.3 Preserve nonstandard models . . . 8

4.4 Why this is separate from previous tag-dispatch advice . . . 8

5 Preservation and fusion for internal recursive graphs **8**

5.1 Three existing clients have the same proof . . . 8

5.2 The preservation theorem . . . 9

5.3 The definability requirement is part of the API 9
5.4 Fusion is the next reusable law 10

6 Synthesize representation-depth bounds with small certificates 10
6.1 The target is depth, rather than polynomial degree 10
6.2 A two-number certificate 10
6.3 Reproducing the existing numerical reasoning 11
6.4 What is still needed to save work 11

7 An optional structural alternative: explicit cyclic carries 11
7.1 Why investigate it, and why keep the existing theorem 11
7.2 Definitions and statement 12
7.3 Proof of existence from one rotation identity 12
7.4 Proof of extremality and the two-solution boundary 13
7.5 The budget singles out the least carry 13
7.6 Validation and formalization boundary 13

8 Adoption order, measurement, and limits 14

9 Archive contents and reproduction 14

1 What was reviewed and what is actually new

1.1 The source baseline

The review uses commit `8e8b5dcb954969621c484588f0b902be1cfea7b3`. The source inventory contains 574 Lean files and 77,278 physical lines under the requested `Lean` directory. Of these, 182 files and 5,380 lines are in the `checks/` subdirectory; 392 files and 71,898 lines are outside that subdirectory. These are file-system counts, not counts of independent theorems or substantive proofs. The inventory records every path, byte count, line count, and SHA-256 digest. The toolchain and lockfile were read directly from the same source snapshot.

The largest files are generated circuit material, but the previous reports already address that material extensively. A fresh review should not simply select the largest files and recommend their already proposed certificate or code-generation refactors again. The six proposals below instead target specific duplicated proof arguments and missing semantic interfaces.

1.2 Nonduplication audit

The current incoming directory contains its README and two ZIPs. The following were read as data, including their substantive supporting notes and proposal files:

1. `diophantine-lean-simplification-2026-10-26.zip`: 48 members.
2. `lean-proof-simplification-2026-10-26.zip`: 64 members.
3. The retired packet under `research/lean-proof-simplification-20261026/`, directly linked by the incoming README. Its report and detailed checker, circuit, and digit notes were inspected.

The archive's `evidence/prior_ideas.md` contains 43 excluded proposal families with precise manuscript or note locations. The two ZIP inventories contain hashes for all 112 members. This matters because several prior recommendations occur only in supplementary notes, such as the origin-decrement word lemma and canonical window-witness extraction.

The scope is intentionally precise: the proposals below were *not found in these audited proof-simplification materials*. This is not a claim of priority across Lean literature, or an assertion that every manuscript transitively linked from the large historical intake index has been read. The mathematical techniques themselves, such as strengthening an induction invariant, are standard. Their contribution here is a concrete application to proof work that the audited packets do not address.

Already proposed; excluded	Distinct target in this report
Generic scanner simulation	Exact image and fuel theorem for the recursive `Tree.decode` syntax decoder
Pell source equations and casts	A transitive signed-congruence relation that absorbs repeated sign choices
Raw tag dispatch and Boolean reflection	Constructor inversion inside arbitrary set-theory models, followed by one definable unique-existence induction
Transport of `omega_ind` through a formula specification	Preservation and fusion of a recursive graph, using totality, functionality, and the successor law
Polynomial degree and monomial majorants	Max-plus propagation of representation *depth* through supplied graph constructors
Carry decomposition and remainder projections	Explicit least/greatest cyclic carries and a complete uniqueness classification

Table 1: The distinction from nearby prior recommendations.

A tempting additional proposal was discarded during the source review: support-restricted evaluation congruence already exists for natural expressions as `WitnessCompaction.eval_congr`. Adding that general idea to the list would have overstated the gap in the existing library.

1.3 Evidence levels

There are four separate kinds of evidence in this delivery:

- **Source observation:** a theorem, repeated branch structure, or numerical bound found at the pinned commit.
- **Checked mechanism:** a standalone theorem checked by Lean 4.32.0, with its actual axiom report retained.
- **Proposed production replacement:** an interface or patch whose full project integration still requires compilation and review.
- **Mathematical research extension:** an ordinary proof and finite experiments, without a completed Lean formalization.

The project Mathlib dependency closure was not available for a full production rebuild in this review. The standalone checks do not replace such a build. No claim of project-wide speedup or total line reduction is made.

2 Fuse syntax-decoder correctness into one graph theorem

2.1 Concrete duplication

In `AnnotatedProofCode.lean`, L78-L132, `Tree.decoded_depth_le` and `Tree.quote_of_decode` independently perform essentially the same analysis: induction on fuel, a zero-code case, an unpairing case, and both recursive `Option` results. One proof recovers the tree's depth bound; the other recovers its original numeric code.

The natural invariant is their conjunction:

$$\mathrm{decode}(f, c) = \mathrm{some}(t) \quad \Longrightarrow \quad \mathrm{quote}(t) = c \ \wedge \ \mathrm{depth}(t) \leq f.$$

Prove this once as `decode_sound`, then retain the old public theorems as its two projections. This is a change to the proof organization only; the concrete decoder, its fuel convention, and its quotation arithmetic remain the same.

2.2 The stronger public specification

The existing `decode_quote` theorem supplies the converse, so the most useful endpoint is

$$\mathrm{decode}(f, c) = \mathrm{some}(t) \quad \Longleftrightarrow \quad c = \mathrm{quote}(t) \ \wedge \ \mathrm{depth}(t) \le f. \tag{1}$$

Either orientation of the equality is harmless; the supplied code uses `quotet=c`. Equation (1) simultaneously records the decoder's exact image and exact sufficient fuel condition. It is stronger and more reusable than keeping two disconnected consequences of a successful decode.

For example, decode monotonicity in fuel follows without another parser induction: recover quotation and depth from success, increase the depth bound, and apply the converse. Rejection below a quoted tree's depth follows from the same theorem. Quotation injectivity can continue to use the existing parser argument; there is no need to alter it merely to increase the apparent size of the refactor.

2.3 Why the proof is smaller

The fork branch already knows that both subdecoders succeeded. Under the joint induction hypothesis it receives

$$\mathrm{quote}(a) = c_a, \quad \mathrm{depth}(a) \le f, \qquad \mathrm{quote}(b) = c_b, \quad \mathrm{depth}(b) \le f.$$

The quotation conclusion follows from the existing pairing inverse; the depth conclusion follows from the maximum of the two bounds. The structural case analysis is paid for once.

The proposed patch replaces a 56-line source region by 38 lines, an 18-line local reduction. This is a physical source measurement of the patch, including its compatibility wrappers, not a benchmark or a claim about compiled proof-term size. The patch applies cleanly to the pinned source. Its production module has not been rebuilt with Mathlib in this review.

2.4 Checked artifact and integration step

`lean/DecoderGraph.lean` checks the general theorem for an explicit one-step coding interface. The interface requires inspection to agree with the atom and fork constructors and requires successful inspection to reconstruct the original code. These are genuine hypotheses, not hidden trust in an external decoder. Its abstract `Code` type also makes clear that the proof mechanism is independent of square-pairing details.

The integration task is small: compile the supplied `DecoderFusion.proposed.patch`, inspect the existing downstream decoder checks, and add the exact graph equivalence as a corollary if it improves clients. The public wrapper names already preserve downstream source compatibility. The separate `ExactDecoderFusion.lean` prototype additionally checks the exact production decoder definitions, arithmetic offsets, and patched pairing/Option proof shape under the explicit pairing-inverse law. That prototype passed Lean 4.32.0 with only `propext` and `Quot.sound` reported.

3 Treat Pell sign ambiguity as a compositional relation

3.1 The source target

In NativePellStepDown.lean, L95-L157, the end of `index_residues` constructs alternatives involving $(-1)^h$, then proves four divisibility cases. The next theorem, `signed_index_residues`, splits the sign of `Int.natAbs` again. The mathematical information being transported throughout is equality of residues *up to sign*.

For a modulus m, define

$$a \sim_m b \quad :\Longleftrightarrow \quad a \equiv b \pmod{m} \ \vee \ a \equiv -b \pmod{m}.$$

This is an equivalence relation and it is unchanged by negating either argument. It is the equivalence generated by ordinary congruence and the involution $x \mapsto -x$. An actual quotient type is unnecessary: the relation already gives the desired proof interface.

3.2 The proof reduction

The source supplies three pieces of information after its substantive Pell argument:

$$u \sim_c J, \qquad u \sim_c s, \qquad s \sim_c p.$$

Then the entire sign bookkeeping becomes

$$J \sim_c u \sim_c s \sim_c p.$$

The second premise follows from $u \equiv (-1)^h s \pmod{c}$ and the elementary two-value property of $(-1)^h$. The third is the existing step-down conclusion, converted from divisibility once. Sign alternatives should be exposed again only at the existing half-modulus theorem, which proves equality of the positive bounded indices.

Likewise, $|V| \sim_c V$ immediately transports the signed auxiliary-coordinate hypothesis. The proof should not rediscover the sign of V every time a residue is used.

Proposition 3.1. *Let R be an equivalence relation preserved by an involution ν. Then $S(a,b) := R(a,b) \vee R(a,\nu(b))$ is an equivalence relation, and replacing either argument by its image under ν preserves S.*

Proof. Reflexivity is immediate. For symmetry, if $R(a,\nu b)$, apply ν to obtain $R(\nu a, b)$, then use symmetry. For transitivity there are four sign combinations; applying invariance under ν to the second relation reduces each to transitivity of R. Two negative signs cancel by involutivity. These four cases belong in the library lemma, rather than in every Pell client. □

3.3 A precise limitation

One must not replace the number-theoretic step-down argument by the assertion that equal squares imply congruence up to sign. For composite moduli this is false: modulo 8, $1^2 \equiv 3^2$, while 1 is congruent to neither 3 nor -3. The new relation simplifies the *transport of already justified residues*; the existing Pell classification and step-down hypotheses remain essential.

At the final equality step, the current positive half-modulus bounds are useful for exactly the right reason. If $0 < p, J < c/2$, the alternative $c \mid J + p$ is impossible, and $c \mid J - p$ implies $J = p$. There is no need to weaken or replace those conditions.

3.4 Checked artifact and expected payoff

`lean/SignOrbit.lean` checks the general involution construction, a concrete integer instance using equality of Euclidean remainders, the three-link residue transport, and transport through `natAbs`. The integer instance makes the artifact more than a purely propositional sketch. Its bridge to Mathlib's `Int.ModEq` and the final divisibility interface remain small integration tasks.

The generic relation proofs report no axioms; the concrete integer instance and its two transport theorems report only `propext`. These are the actual reported dependencies. No `sorry`, added axiom declaration, or native evaluation was used. The proposed production saving has not been measured because the Pell file has not been patched and rebuilt.

4 Constructor views and one internal unique-existence induction

4.1 Expose the meaning of a known constructor once

`ObjectCodeBound.BoundStep` is an eight-way disjunction: two atomic tags, bottom, three binary tags, and two quantifier tags. The four inversion theorems at `ObjectCodeBound.lean`, L240-L311 repeatedly open this disjunction, reject incompatible tags, apply injectivity of nested `kpair`, and transport the surviving payload.

Introduce a reusable constructor view. Abstractly, if $T : \mathbb{N} \to V$ is injective and $K : V \times V \to V$ is an injective pairing operation, then

$$K(T(s), a) = K(T(t), b) \quad \iff \quad s = t \ \wedge \ a = b.$$

For a tagged relation

$$\mathcal{R}(c, z) \iff \exists t, p, \ A(t) \wedge c = K(T(t), p) \wedge L(t, p, z),$$

one gets the constructor-specific view

$$\mathcal{R}(K(T(t), p), z) \quad \iff \quad A(t) \wedge L(t, p, z).$$

Prove equivalence between the existing eight-way `BoundStep` and such a presentation once. Keep the existing definition and its object-language formula if they are useful for literal provenance. The view is a theorem about that definition.

The binary specialization should expose, for $t \in \{3, 4, 5\}$, exactly

$$\exists a', d', \quad K(a, a') \in S \ \wedge \ K(d, d') \in S \ \wedge \mathrm{AddRel}(a', d', b).$$

The atom specialization must retain the naturalness conditions on its indices; suppressing them would change its meaning. The quantifier specialization simply exposes the child pair in S. This gives each inversion client a small, precise theorem instead of another eight-way analysis.

4.2 Strengthen the induction from totality to unique existence

The current `boundCodeRel_total` and `boundCodeRel_unique` perform separate internal formula-code inductions at `ObjectCodeBound.lean`, L324-L405. A stronger invariant is

$$\mathrm{IsFormCodeSem}(c) \quad \Longrightarrow \quad \exists! b, \ \mathrm{BoundCodeRel}(c, b). \tag{2}$$

The existing APIs become projections. Totality uses the witness; functionality compares each proposed result to that witness.

For a binary constructor, obtain unique child outputs a' and d' from the two induction hypotheses. Obtain a sum output b from internal addition and construct the parent certificate. To prove uniqueness, take one arbitrary competing parent result, invert it *once*, identify its child outputs by the induction hypotheses, and use addition functionality. The old uniqueness proof inverts two arbitrary parent results; the strengthened invariant has already constructed a canonical one.

The internal formula for this invariant is concrete. With the code in the current slot zero, it can be expressed by the following existing object-language constructors:

```
fEx (fAnd (boundCodeForm 0 1)
  (fAll (fImp (boundCodeForm 0 2) (fEq 0 1))))
```

After the existential binder, the chosen output is slot 0 and the code is slot 1. After the universal binder, the competing output is slot 0, the chosen output is slot 1, and the code is slot 2. The displayed indices therefore match unique existence, including equality to the chosen witness.

4.3 Preserve nonstandard models

This proposal must use the existing `isFormCodeSem_ind'` principle with an actual formula specifying (2). It is not sound to replace that induction with an external inductive datatype whose constructors enumerate only standard finite formulas. Nor should one assume that every internal code is the quotation of an external Lean value. The existing source explicitly covers nonstandard codes; the refactor must retain that scope.

The checked `lean/TaggedConstructor.lean` works over an arbitrary carrier and assumes only tag and pairing injectivity. It proves constructor discrimination and a generic tagged-relation inversion theorem. The full `BoundStep` equivalence and the combined object-level unique-existence proof are proposed production work. The artifact does not claim to have formalized them already.

4.4 Why this is separate from previous tag-dispatch advice

The earlier tag-dispatch proposals concern raw numeric checkers and their Boolean outputs. Here the purpose is to eliminate semantic disjunctions over the model's set-valued codes and support internal definable induction. The constructor tags may be standard numerals while the payload is an arbitrary internal object. That distinction is exactly why the arbitrary-carrier interface is appropriate.

5 Preservation and fusion for internal recursive graphs

5.1 Three existing clients have the same proof

The following theorems all construct a stage predicate, invoke definable induction over internal omega, use totality to recover a previous result, and use functionality to identify the successor result:

- `ObjectArithmetic.lean`, L103-L126: `addRel_result_nat`.
- `ObjectMultiplication.lean`, L130-L151: `mulRel_result_nat`.
- `ObjectCodeClosure.lean`, L94-L112: `closeCodeRel_isFormCode`.

The common argument belongs at the level of the recursive graph Θ_G, not in a generic tactic that happens to reproduce each script.

5.2 The preservation theorem

Fix a step function F, seed a, and a recursive graph $G(a, z, n)$. Assume:

1. Every graph instance has an admissible internal index; at every such index the graph is total and single-valued.
2. $G(a, a, 0)$, and $G(a, z, n)$ implies $G(a, F(z), \mathrm{succ}(n))$.
3. The precise stage predicate $I(n) := \forall z,\ G(a, z, n) \to P(z)$ has the model's valid definable induction principle.
4. $P(a)$, and $P(z) \to P(F(z))$.

Then

$$G(a, z, n) \implies P(z).$$

Proof. At zero, functionality identifies the arbitrary result with a. At a successor stage, totality supplies w at the preceding index. The induction hypothesis gives $P(w)$. The successor law produces $F(w)$ at the desired index, and functionality identifies the arbitrary result z with $F(w)$. Step closure gives the conclusion. □

The three clients then supply their genuine mathematical content:

Client	Invariant	Local closure fact
Addition	Output belongs to internal omega	Internal successor preserves omega
Multiplication	Output belongs to internal omega	Adding the fixed internal-natural parameter preserves omega
Code closure	Output is an internal formula code	Adding a universal quantifier preserves internal formula codes

5.3 The definability requirement is part of the API

For an arbitrary model, an unrestricted external predicate on its internal omega need not satisfy induction. A correct Lean theorem must require a concrete formula and a satisfaction equivalence for I, or an already established induction proof for this exact predicate. The supplied `IterationGraph.lean` uses an arbitrary index type and a parameter `InductionForD(PreservedStage...)`. It does not identify that index type with Lean's `Nat` and does not assert induction for every predicate.

For production use, the actual `ThetaG` totality theorem also needs a definable step relation. The adapter should retain that formula and its specification. A generic constructor for the stage formula can eventually remove the three repeated `stageNaturalForm_spec`/`stageCodeForm_spec` proofs, but that formula-generation layer is a separate implementation obligation. Merely hiding a supplied stage proof inside a record does not discharge it.

The earlier proposal `omega_ind_of_spec` can serve as infrastructure. The new result goes further: it proves the recursive-graph preservation argument using totality, uniqueness, and the successor equation, leaving clients only their seed and closure facts.

5.4 Fusion is the next reusable law

For recursive graphs of F and G, if a map h satisfies

$$h(F(x)) = G(h(x)),$$

then the corresponding graph-level transport is

$$\Theta_G(F, a, z, n) \implies \Theta_G(G, h(a), h(z), n).$$

Its proof uses the same induction pattern with the displayed graph-transport predicate. This is useful for commuting arithmetic or coding transformations through an internal iteration. Production fusion must again retain definability of the exact invariant and any represented maps used to construct it.

The standalone file checks both preservation and fusion, with no axioms reported for either theorem. Its hypotheses are explicit; it is evidence for the general argument, not an integration proof for the current `ThetaG` implementation.

6 Synthesize representation-depth bounds with small certificates

6.1 The target is depth, rather than polynomial degree

`BoundedForallDepth.lean`, L122-L189 contains named inequalities for almost every intermediate graph in `factorial_depth` and `product_depth_le`. Most of this work propagates maximum and addition bounds, lifting the smaller input bound to a common larger one. The later `finiteRep_depth_le` repeats the same style on a larger constructor expression.

The source already provides the necessary local depth lemmas. The proposed improvement is a small proof-producing interpreter for their numerical consequences, without expanding the represented arithmetic graph or polynomial expression. It is distinct from the prior weighted polynomial-degree and coefficient-majorant proposals.

6.2 A two-number certificate

Associate to a constructor plan a pair (r, e): a required baseline r, and an excess depth e. Its meaning is:

$$h \geq r \quad \implies \quad \text{result depth} \leq h + e,$$

assuming all leaves have depth at most h.

For a binary local bound

$$d \leq \max(f, \max(d_1, d_2)) + c,$$

combine child summaries by

$$r = \max(f, r_1, r_2), \qquad e = c + \max(e_1, e_2).$$

For a unary local bound, use $r = \max(f, r_1)$ and $e = c + e_1$. A leaf has summary $(0, 0)$. This is the maximum-and-addition arithmetic appropriate to depth.

`lean/DepthBudget.lean` defines a certificate relation `Covers`. The relation connects a plan to a numerical depth through those local inequalities. Its soundness theorem proves the bound above. It does traverse a plan tree; it does *not* expand the graph or polynomial syntax whose depth is being bounded.

6.3 Reproducing the existing numerical reasoning

The supplied plans follow the actual source constructor shapes. An independent comparison checked the primitive charges and intermediate offsets:

Plan	Required baseline	Excess depth
Factorial constructor expression	512	50
Product constructor expression	16,384	66

Thus the factorial plan recovers $512+50 = 562$, exactly the intermediate bound already established before the source relaxes it to 16,384. This report does not claim a new numerical improvement to that theorem. The product plan recovers the existing $h + 66$ bound. The arithmetic composite constructors have charge 8; addition/multiplication have charge 6; the factorial graph wrapper has charge 6 with baseline 16,384. Those are source-specific adapter facts, not universal claims about all graph representations.

6.4 What is still needed to save work

A handwritten plan plus a handwritten `Covers` derivation would only move the existing proof work. The useful implementation must do the following:

1. Supply one local adapter for each actual `GraphRep` constructor lemma.
2. Traverse the source constructor expression, respecting local `let` sharing, and build the certificate proof automatically.
3. Memoize shared subterms and retain opaque or frozen semantic boundaries; do not unfold a whole representation record to recover an elementary depth bound.
4. Let the kernel check the produced proof; finish the requested numerical target by a small arithmetic step.

The checked artifact supplies the numerical soundness theorem and the two source-shaped plans. The actual `GraphRep`-to-`Covers` adapters and traversal are not implemented. `finiteRep_depth_le` is a proposed later client, not a tested conversion. This is a medium-sized automation investment whose success criterion is removal of manual bound plumbing from real clients, rather than the mere existence of a separate certificate language.

7 An optional structural alternative: explicit cyclic carries

7.1 Why investigate it, and why keep the existing theorem

`NativeCyclicCarry.lean`, L37-L47 already proves existence of bounded carries very concisely using a least fixed point. Its predecessor map is arbitrary, and it allows any positive radix. Replacing that theorem wholesale would lose useful generality and is unlikely to shorten this local proof.

The distinct opportunity is for a *genuine finite cycle* with radix $\rho > 1$: explicitly compute the least and greatest carries and characterize the only possible ambiguity. This could simplify future exact-witness and uniqueness arguments in `NativeCyclicCarryWord` and its clients. It should be adopted only if those clients justify the new infrastructure.

7.2 Definitions and statement

Let $n > 0$, $\rho > 1$, and let $a_i \in \mathbb{N}$, with indices in $\mathbb{Z}/n\mathbb{Z}$. We seek

$$k_i = \left\lfloor \frac{a_i + k_{i-1}}{\rho} \right\rfloor. \tag{3}$$

Put

$$M = \rho^n - 1, \qquad A_i = \sum_{j=0}^{n-1} \rho^{n-1-j} a_{i-j}.$$

Define

$$k_i^- = \frac{A_i \mathbin{\dot{-}} 1}{M} \text{ (natural quotient)}, \qquad k_i^+ = \left\lfloor \frac{A_i}{M} \right\rfloor, \tag{4}$$

where $\dot{-}$ is truncated natural subtraction. Both formulas give zero when all raw digits vanish.

Theorem 7.1 (Cyclic carry classification)**.** *Both k^- and k^+ satisfy (3). Every nonnegative solution lies componentwise between them. If the raw word is zero, the solution is unique. If it is nonzero and $M \nmid A_i$ for some (equivalently every) index i, the solution is also unique. Otherwise there are exactly two global solutions: k^- and k^+, which differ by one in every coordinate. Their normalized digit words are respectively all $\rho - 1$ and all zero.*

7.3 Proof of existence from one rotation identity

The fundamental identity is

$$\rho A_i = M a_i + A_{i-1}. \tag{5}$$

It is a direct index shift in the weighted sum.

For the greatest formula, write $A_i = M k_i^+ + r_i$, with $0 \le r_i < M$. Substitution in (5) gives

$$M(a_i + k_{i-1}^+ - \rho k_i^+) = \rho r_i - r_{i-1}.$$

The right side is strictly greater than $-M$ and strictly less than ρM. The left side is an integer multiple of M. Therefore

$$0 \le a_i + k_{i-1}^+ - \rho k_i^+ < \rho,$$

which is precisely the quotient recurrence.

For the least formula, first separate the all-zero raw word. Otherwise every $A_i > 0$, and one can use *positive* remainders

$$A_i = M k_i^- + e_i, \qquad 1 \le e_i \le M.$$

Now the same identity gives

$$M(a_i + k_{i-1}^- - \rho k_i^-) = \rho e_i - e_{i-1}.$$

Again the right side is strictly between $-M$ and ρM, hence the quotient recurrence follows. The use of positive rather than conventional remainders selects the least carry at the exceptional zero residue.

7.4 Proof of extremality and the two-solution boundary

Let k be any solution and put $d_i = a_i + k_{i-1} - \rho k_i$. The recurrence gives $0 \le d_i < \rho$. Multiply the local equations by the same rotated powers and sum around the entire cycle. The carry terms telescope, giving

$$A_i = Mk_i + D_i, \qquad D_i = \sum_{j=0}^{n-1} \rho^{n-1-j} d_{i-j}, \qquad 0 \le D_i \le M.$$

Consequently

$$Mk_i \le A_i \le M(k_i + 1). \tag{6}$$

These inequalities put k_i between (4). If A_i is not divisible by M, there is only one integer possibility. If $A_i = q_i M > 0$, there are two: $q_i - 1$ and q_i.

Since $\gcd(\rho, M) = 1$, identity (5) shows that divisibility by M propagates around the cycle. In the divisible nonzero case, write $k_i = q_i - \varepsilon_i$, where $\varepsilon_i \in \{0, 1\}$. The recurrence and $\rho q_i = a_i + q_{i-1}$ imply $\varepsilon_i = \varepsilon_{i-1}$. Thus independent local sign choices do not create 2^n solutions: all choices synchronize, leaving exactly two global solutions. Substitution gives digits zero for k^+ and digits $\rho - 1$ for k^-.

If all $A_i = 0$, inequality (6) forces all carries to be zero. This includes the small boundary $n = 1, \rho = 2, M = 1$; the nonzero case still has the two solutions described above.

7.5 The budget singles out the least carry

The current bounded-carry hypothesis is

$$a_i + D < (D + 1)\rho.$$

Over natural numbers this implies $a_i \le (D + 1)(\rho - 1)$, hence

$$A_i \le (D + 1)(\rho - 1) \sum_{j=0}^{n-1} \rho^j = (D + 1)M.$$

For a nonzero word, $A_i - 1 < (D + 1)M$, so $k_i^- \le D$. The zero case is immediate. The greatest formula may instead equal $D + 1$. For example, when every $a_i = (D + 1)(\rho - 1)$, the budget holds, the least carry is constantly D, and the greatest carry is constantly $D + 1$. A replacement bounded-witness construction must choose the least formula.

If an existing transport premise identifies the raw word with a strict residue $0 < F < M$, its residue is nonzero. The exceptional two-solution case is therefore excluded. The current `transport_word_eq` already identifies the normalized word; the new theorem would additionally give uniqueness of the carry assignment and an explicit canonical witness. These are additional conclusions, not a reason to restate the existing word theorem as if it were absent.

7.6 Validation and formalization boundary

The supplied Python experiment checks 57,540 raw-array/budget instances for $2 \le \rho \le 5$, $1 \le n \le 4$, and $0 \le D \le 2$. In 2,323 smaller instances it enumerates every carry vector in the tested box and checks that the complete solution set is exactly $\{k^-, k^+\}$. It also compares the least formula with monotone iteration from zero. There are 57,308 tested instances with nonzero residue, in which the two formulas coincide.

These are finite checks, not a Lean formalization or a substitute for the proof above. The implementation should begin with (5), then the positive-remainder lemma and the telescoping identity. Keep the existing arbitrary-predecessor fixed-point theorem as the general fallback. No unconditional uniqueness claim is valid.

8 Adoption order, measurement, and limits

Step	Proposal	First concrete milestone	Evidence in this delivery
1	Decoder graph fusion	Compile the proposed patch and preserve the two public projection theorems	Applicable patch; checked generic mechanism; measured 18-line local reduction
2	Signed congruence	Instantiate the API for `Int.ModEq`; replace the final sign-transport block	Checked abstract and concrete integer transport
3	Constructor views and unique existence	Prove `BoundStep` views; prove one definable ∃! theorem	Checked arbitrary-carrier inversion mechanism; explicit object formula
4	Internal graph preservation	Adapt `ThetaG`; convert the three naturalness/code-preservation clients	Checked preservation/fusion with exact invariant-induction premise
5	Depth certificates	Implement constructor adapters and automated certificate extraction; convert factorial and product	Checked numerical checker; two matching source plans; adapter still absent
6	Explicit cyclic carries	Formalize rotation/telescoping identities; demonstrate a downstream uniqueness or witness benefit	Ordinary proof, peer review, finite experimental checks; no Lean formalization

For the first four directions, measure source size after including new shared lemmas, not merely the shortened client. Keep executable definitions, object-language formulas, and public endpoints stable unless a separate reason justifies changing them. For depth automation, measure both the client script and the generated proof-checking cost; source brevity alone does not establish performance. For cyclic carries, require a concrete downstream payoff before replacing any existing proof.

The acceptance checks should be narrowly relevant: compile affected modules and their existing endpoint checks, compare the unchanged public theorem statements, and inspect axiom reports. The standalone logs record only the listed prototype theorems. They do not establish the axiom profile of a future production patch. Historical compiler receipts inside prior packets are not counted as fresh validation here.

9 Archive contents and reproduction

The ZIP contains the article in PDF and LaTeX, six Lean prototypes, the proposed decoder patch, and reproduction scripts.

The `evidence/` directory contains the source inventory, incoming-archive hashes, exclusion matrix, compiler logs, numerical experiment output, and independent reviews. `README.txt` states the precise validation status of each file. The package does not contain a full checkout of the repository or Mathlib.

Use Lean 4.32.0 for the standalone files. The verification script accepts an explicit compiler path and records its version, exit codes, output, and source hashes. The decoder patch can be checked against the pinned checkout before any application. To rebuild the manuscript, run the provided PDF build script with a normal TeX installation. To repeat the carry experiment, run the Python script; it has no third-party package dependency and does not execute repository or archived code.

Final assessment

The most attractive immediate changes are the decoder conjunction invariant and the signed-congruence interface: both remove repeated analysis with little semantic disturbance. The constructor-view and internal-recursion proposals offer a larger reduction in repeated reasoning while preserving the project's essential treatment of nonstandard models. The depth checker is worthwhile only when its adapters and traversal automate actual clients. The cyclic-carry formula is the most substantial mathematical alternative in this report, but its engineering value depends on new exactness or uniqueness clients; the existing general existence proof is already efficient.

References

[1] Vladimir Reshetnikov, *ProveIt*, reviewed commit `8e8b5dcb954969621c484588f0b902be1cfea7b3`. https://github.com/VladimirReshetnikov/ProveIt.

[2] *Incoming research material*, current ZIPs and intake routing. https://github.com/VladimirReshetnikov/ProveIt/tree/8e8b5dcb954969621c484588f0b902be1cfea7b3/docs/incoming.

[3] *Placed DiophantineConsistency proof-simplification packet*. https://github.com/VladimirReshetnikov/ProveIt/tree/8e8b5dcb954969621c484588f0b902be1cfea7b3/Computability/DiophantineConsistency/research/lean-proof-simplification-20261026.

[4] Primary Lean sources are linked at the exact theorem locations throughout this article. All local source hashes appear in `evidence/source_inventory.json`.

[5] Standalone formal and experimental evidence produced for this review: `evidence/validation.json`, the individual compiler logs, and `evidence/cyclic_carry_checks.json`. Their scopes are stated in the relevant sections above.